IT DevOps Engineer Case – Automatisierung kritischer IT-Prozesse

Ausgangssituation

Du bist als IT DevOps Engineer bei einem wachsenden Technologieunternehmen tätig. Das Unternehmen expandiert schnell, und die IT-Abteilung steht vor mehreren kritischen Herausforderungen bei der Skalierung ihrer Prozesse. Zwei besonders dringende Problembereiche wurden identifiziert, die beide erhebliche Auswirkungen auf die Effizienz und Zufriedenheit der Mitarbeitenden haben.

Deine Situation

Das Management hat dir zwei kritische Projekte zugeteilt, die beide höchste Priorität haben. Aufgrund begrenzter Ressourcen und der Dringlichkeit der Situation musst du dich für eines der beiden Szenarien entscheiden und dieses vollständig ausarbeiten. Beide Projekte adressieren zentrale Herausforderungen der Organisation und bieten erhebliches Potenzial für Verbesserungen.

Wähle eines der folgenden Szenarien aus und bearbeite es vollständig:

Szenario A: Automatisierung des Mitarbeiter-Onboardings

Problembeschreibung

Der aktuelle Prozess für das Onboarding neuer Mitarbeitender ist stark manuell geprägt und skaliert nicht mit dem Wachstum des Unternehmens.

Aktueller Prozessablauf

- HR-System: Nach der Vertragsunterzeichnung legt die Personalabteilung (HR) manuell eine neue Person im zentralen HR-System an.
- Benachrichtigung: HR exportiert eine PDF-Zusammenfassung und sendet diese per E-Mail an das IT-Team. Betreff: „Neueinstellung: [Name]“.
- Manuelle Account-Erstellung: Ein IT-Mitarbeitender arbeitet die E-Mail ab und führt folgende Schritte durch:
    - Erstellung eines Active-Directory-(AD)-Accounts
    - Zuweisung zu Standard-AD-Gruppen für den Zugriff auf Netzlaufwerke
    - Erstellung eines Microsoft-365-Accounts (inkl. E-Mail-Postfach)
    - Manuelles Anlegen von Accounts in weiteren Kernsystemen (z. B. Projektmanagement-Tool und Wiki)

- Hardware-Provisionierung: Parallel dazu werden Bestellung und Einrichtung eines Notebooks in einer separaten Excel-Liste erfasst und nachverfolgt.
- Zugangsübergabe: Initiale Passwörter und Benutzernamen werden in einem Dokument erfasst und der neuen Person am ersten Tag in einem verschlossenen Umschlag übergeben.

Kernprobleme

- Fehleranfälligkeit: Manuelle Datenübertragung führt regelmäßig zu Fehlern (z. B. Tippfehler in Namen, falsche Gruppenzuweisungen).
- Mangelnde Transparenz: Niemand hat einen zentralen Überblick über den Status des Onboarding-Prozesses.
- Hohe Durchlaufzeit: Neue Mitarbeitende sind oft erst am Nachmittag ihres ersten Tages voll arbeitsfähig.
- Sicherheitsrisiken: Die Verwaltung von initialen Passwörtern ist umständlich und birgt Risiken.
- Keine Skalierbarkeit: Bei mehreren Neueinstellungen pro Woche wird das IT-Team zum Flaschenhals.

Deine Aufgabe für Szenario A

Entwirf eine robuste und skalierbare Lösung zur vollständigen Automatisierung des IT-Onboarding-Prozesses. Du sollst diese Lösung im nächsten Gespräch innerhalb von 20–40 Minuten präsentieren. Strukturiere deine Präsentation anhand der folgenden vier Kernbereiche:

1) Lösungsarchitektur & Technologie-Auswahl

- Visualisiere den automatisierten Zielprozess, den du entworfen hast.
- Welche zentrale Technologie oder Plattform schlägst du als Herzstück der Automatisierung vor? Begründe deine Wahl im Hinblick auf Flexibilität, Wartbarkeit, Kosten und Anforderungen.
- Wann würdest du auf eine SaaS-Lösung vs. Self-Hosted vs. Hybrid setzen? Begründe anhand konkreter Szenarien.
- Skizziere die Architektur: Wie interagieren HR-System, Active Directory und weitere Applikationen mit der zentralen Lösung?

2) Technische Umsetzung & Sicherheit

- Erläutere den technischen Aufbau der Kernlogik. Wie würdest du die einzelnen Schritte implementieren und orchestrieren?

- Wie stellst du sicher, dass der Prozess robust ist? Beschreibe dein Konzept für Error-Handling, das Vorgehen bei Ausfall eines Zielsystems und wie du idempotente Operationen gewährleistest.
- Wie gehst du mit sensiblen Informationen wie API-Keys, Service-Account-Credentials und insbesondere den initialen Passwörtern für neue Mitarbeitende um?

3) Monitoring, Deployment & Betrieb

- Welches Konzept für Monitoring, Logging und Alerting schlägst du vor, um die Prozessstabilität und eine lückenlose Nachvollziehbarkeit jedes Onboarding-Vorgangs zu gewährleisten? Welche Metriken wären dir wichtig?
- Beschreibe den Deployment-Prozess für deine Lösung. Wie würdest du Änderungen am Automatisierungsprozess entwickeln, testen und produktiv setzen?
- Erstelle exemplarisch ein Terraform-, Pulumi- oder vergleichbares Infrastructure-as-Code-Script für ein mögliches Cloud-Deployment (AWS/Azure/GCP) deiner Lösung.
  Hinweis: Ein tatsächliches Deployment in der Cloud ist NICHT erforderlich.
  Begründe deine Dimensionierung der Cloud-Ressourcen basierend auf erwarteten Onboarding-Volumes und Spitzenlasten.

4) Erweiterbarkeit & nächste Schritte

- Wie stellst du in deinem Design sicher, dass die Lösung einfach erweiterbar ist, um zukünftig weitere Systeme (z. B. eine Zeiterfassung) anzubinden oder komplexere, abteilungsspezifische Logik (z. B. „Entwickler erhalten zusätzlich Zugriff auf GitHub“) abzubilden?
- Skizziere kurz, wie du dieses Projekt in der Praxis angehen würdest.

Szenario B: Self-Service-Portal für Berechtigungsanfragen

Problembeschreibung

Aktuell beantragen Mitarbeitende den Zugriff auf spezielle Ressourcen (z. B. Lizenzen für Fachsoftware, Zugriff auf Projekt-SharePoints) per E-Mail bei ihrer Führungskraft. Diese leitet die E-Mail an das IT-Team weiter, das die Berechtigung anschließend manuell im jeweiligen System (z. B. in einer Active-Directory-Gruppe) einträgt.

Kernprobleme des aktuellen Prozesses

- Intransparent: Anfragen gehen unter oder bleiben in Postfächern liegen; der aktuelle Status ist für niemanden ersichtlich.
- Langsam: Mehrstufige manuelle Bearbeitung führt zu langen Wartezeiten.
- Nicht nachvollziehbar: Es gibt keine zentrale Dokumentation, wer wann welche Berechtigung genehmigt hat.
- Aufwändig für die IT: Das Team wird mit repetitiven Routineaufgaben belastet.

Deine Aufgabe für Szenario B

Entwickle einen funktionsfähigen Prototyp für ein Self-Service-Portal, über das Mitarbeitende eigenständig den Zugriff auf Ressourcen beantragen können. Der Prototyp soll den gesamten Prozess von der Anfrage bis zur technischen Umsetzung abbilden.

Anforderungen an die Lösung

1) Web-Oberfläche (Frontend)

- Erstelle ein einfaches Web-Formular mit den folgenden Feldern:
    - Ein Dropdown-Menü, um die zu beantragende Ressource auszuwählen (z. B. „Lizenz für Analyse-Software“, „Zugriff auf Projekt-Marketing-SharePoint“)
    - Ein Textfeld für eine kurze Begründung
    - Ein „Senden“-Button
- Die Wahl der Frontend-Technologie ist dir überlassen

2) Workflow & Backend-Logik

- Das Backend soll die Anfrage vom Frontend entgegennehmen
- Es soll die Führungskraft der anfragenden Person identifizieren
  Annahme: Es gibt einen internen API-Endpunkt `GET /api/manager/{user-email}`, der die E-Mail-Adresse der Führungskraft zurückgibt

- An die Führungskraft soll eine automatisierte E-Mail mit dem Inhalt der Anfrage und zwei Links/Buttons gesendet werden: „Genehmigen“ und „Ablehnen“

3) API-Integration & Provisioning

- Wenn die Führungskraft auf „Genehmigen“ klickt, soll das Backend eine Aktion auslösen
- Diese Aktion soll per API-Call die anfragende Person zu einer vordefinierten Berechtigungs-Gruppe hinzufügen

4) Benachrichtigung

- Die anfragende Person soll nach der Entscheidung der Führungskraft (sowohl bei Genehmigung als auch bei Ablehnung) eine kurze Bestätigungs-E-Mail erhalten

Inhalte für die Präsentation (ca. 20–40 Minuten)

- Live-Demonstration: Führe uns live durch den Prozess. Fülle das Formular aus, zeige die (simulierte) Genehmigungs-E-Mail und den erfolgreichen API-Call.
- Code-Walkthrough: Zeige uns die interessantesten Teile deines Codes. Erkläre kurz die Struktur deines Projekts und erläutere:
    - Wie Frontend und Backend kommunizieren
    - Wie der Klick auf den Genehmigungs-Link im Backend verarbeitet wird
- Design- und Technologie-Entscheidungen: Begründe kurz deine Wahl der eingesetzten Technologien (Programmiersprache, Frameworks etc.). Wie handhabst du sensible Daten wie API-Schlüssel oder Credentials in deinem Prototyp?
- Nächste Schritte & Verbesserungen: Wenn du mehr Zeit hättest, was wären die nächsten drei Dinge, die du an deinem Prototyp verbessern oder hinzufügen würdest?

Hinweise für beide Szenarien

- Triff plausible Annahmen**,** falls dir Informationen fehlen, und dokumentiere diese explizit in deiner Präsentation
- Erkläre (wenn notwendig), welche zusätzlichen Informationen du für eine vollständige Umsetzung benötigen würdest
- Es geht nicht darum, ein fertiges Produkt zu zeigen, sondern deine Fähigkeit, ein solches Problem strukturiert zu analysieren und eine technisch fundierte Lösung zu konzipieren bzw. einen funktionierenden Prototypen zu implementieren